Université Versailles-Saint-Quentin

Paris-Saclay



Spécialité :
Calcul Haute Performance, Simulation

Module :
AISE

January 6, 2026

Rapport :
Projet <ExPResSo>

Présenté par: Masten OUNNAS

Contents

1 Introduction 2
1.1 Environnement Matériel 2
2 Choix d'Implémentation de ExPResSO 2
2.1 Gestion des Threads 2
2.2 Structures de Données et Synchronisation 2
3 Stratégies de Parallélisation 3
3.1 Applications Régulières (Vecmul, Dot, Matmul) 3
3.2 Application Irrégulière (List) 3
4 Analyse des Performances 3
4.1 Remarque sur la durée des tests 3
4.2 Passage à l'Échelle (Matmul 1024x1024) 3
4.2.0.1 Observations : 4
4.3 Comparaison sur Charge Irrégulière (Benchmark List) 4
4.3.0.1 Analyse des résultats : 5
4.3.0.2 Discussion critique : 5
5 Conclusion 5

1 Introduction

L'exploitation des architectures multi-cœurs modernes nécessite des modèles de programmation adaptés. Si la bibliothèque *pthreads* offre un contrôle fin, elle reste complexe à manipuler directement. Le modèle *Fork-Join*, utilisé notamment par OpenMP, simplifie cette approche.

Dans le cadre de ce projet, nous avons implémenté **ExPResSO**, un support d'exécution (*runtime*) minimaliste basé sur ce modèle. Nous avons ensuite utilisé cette bibliothèque pour paralléliser plusieurs applications (produit scalaire, multiplication de matrices, traitement de liste). Ce rapport présente nos choix d'implémentation, les stratégies de décomposition adoptées et une analyse des performances obtenues sur notre machine de test.

1.1 Environnement Matériel

Les tests de performance présentés dans ce rapport ont été réalisés sur une machine équipée d'un processeur récent, dont les caractéristiques influencent les résultats (notamment la taille des caches) :

- **Processeur :** AMD Ryzen™ 7 7735HS.
  - Architecture : Zen 3+ ("Rembrandt R"), 6nm.
  - Cœurs : 8 Physiques / 16 Logiques (SMT).
  - Fréquence : Base 3.2 GHz / Boost 4.75 GHz.
  - **Caches :** L1 (32KB+32KB/cœur), L2 (512KB/cœur), **L3 (16 MB partagé)**.
- **RAM :** 16 GB DDR5-4800 MHz.

2 Choix d'Implémentation de ExPResSO

Pour la conception de la bibliothèque, nous avons cherché à limiter le surcoût de gestion des threads (*overhead*).

2.1 Gestion des Threads

Nous avons implémenté un modèle de **Thread Pool**. Les threads workers sont créés une seule fois au démarrage (`expresso_initialize`) et boucle en attendant du travail. Cela évite le coût système des appels `pthread_create` à chaque nouvelle tâche, ce qui est critique pour les applications générant beaucoup de tâches fines.

2.2 Structures de Données et Synchronisation

Pour supporter les différentes politiques d'ordonnancement demandées (Dynamic, Static, Balanced), nous sommes passés d'une file unique à une structure hybride :

- Une **file globale** pour le mode dynamique.
- Un tableau de **files locales** (une par worker) pour les modes statique et équilibré.

Cela permet, dans les modes statiques, que chaque worker consomme sa propre file sans entrer en compétition avec les autres sur un mutex global, réduisant ainsi la contention.

La synchronisation s'appuie sur `pthread_cond_wait` pour mettre les workers en sommeil lorsqu'ils n'ont plus de tâches, évitant l'attente active. Pour la fonction `expresso_wait` (barrière), nous avons ajouté un appel à `sched_yield()` dans la boucle du thread principal pour qu'il relâche le CPU s'il ne trouve pas de tâche à exécuter, laissant la priorité aux workers actifs.

3 Stratégies de Parallélisation

3.1 Applications Régulières (Vecmul, Dot, Matmul)

Pour `vecmul` et `dot`, nous avons découpé les itérations en blocs contigus. Pour **Matmul**, nous avons opté pour une décomposition par **bandes de lignes**. Comme le C stocke les matrices ligne par ligne, traiter des lignes contiguës est préférable pour la localité spatiale et l'utilisation des lignes de cache, contrairement à un découpage par colonnes.

Pour le produit scalaire (`dot`), la réduction finale pose un problème d'accès concurrent. Plutôt que d'utiliser un verrou atomique à chaque addition (trop coûteux), chaque tâche calcule une somme partielle locale. Le thread principal effectue la somme de ces résultats partiels à la fin du calcul.

3.2 Application Irrégulière (List)

L'application `list` présente une charge déséquilibrée simulée par des `sleep()`. Nous ne pouvions pas découper la liste chaînée en blocs sans la parcourir. Nous avons donc choisi de créer **une tâche par nœud**. Bien que cela augmente le nombre total de tâches, cette granularité fine est nécessaire pour que l'ordonnanceur *Balanced* puisse répartir la charge efficacement. Le "poids" de la tâche correspond à la valeur du nœud.

4 Analyse des Performances

Nous concentrons notre analyse détaillée sur les benchmarks les plus représentatifs : `matmul` pour illustrer le passage à l'échelle sur des calculs intensifs, et `list` pour évaluer l'impact des politiques d'ordonnancement

4.1 Remarque sur la durée des tests

L'exécution de la suite de tests (`make test`) prend plusieurs minutes. Ceci est normal et dû au benchmark `list` : pour une liste de 10 éléments, le temps séquentiel incompressible est de 55 secondes. La validation complète confirme que le runtime gère correctement ces temps d'attente sans interblocage.

4.2 Passage à l'Échelle (Matmul 1024x1024)

Nous avons mesuré les temps d'exécution sur les 8 cœurs physiques de la machine.

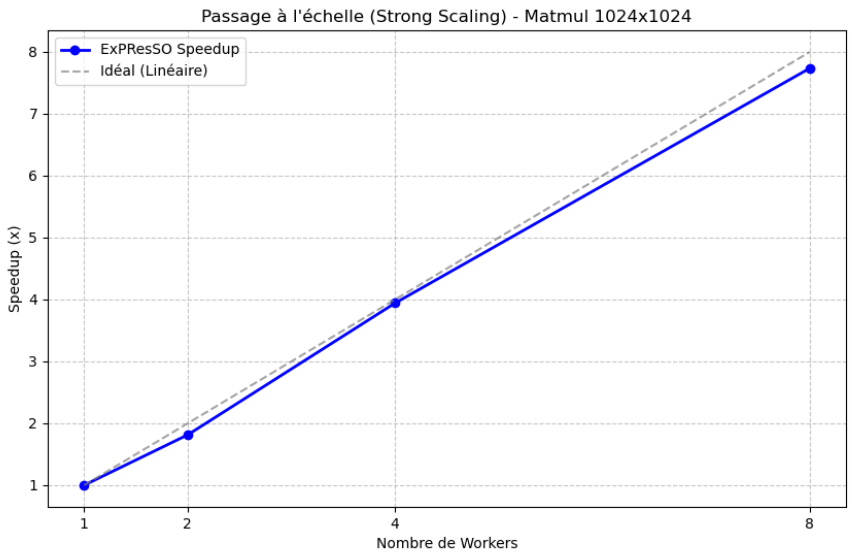


Figure 1: Speedup sur Matmul (1024x1024) - Strong Scaling

Workers	Temps (s)	Speedup (vs 1 Worker)	Efficacité
1	4.10	1.0	100%
2	2.26	1.81	90.5%
4	1.04	3.94	98.5%
8	0.53	7.73	96.6%

4.2.0.1 Observations :

1. **Effet de Cache :** Le code séquentiel de référence s'exécute en 10.4s, alors que ExPResSO avec 1 seul worker met 4.10s. Cet écart s'explique par la décomposition en blocs (tiling implicite) effectuée par nos tâches, qui permet aux données de mieux tenir dans le cache L2/L3 du processeur par rapport à la boucle naïve de référence.

2. **Scalabilité :** De 1 à 8 workers, le speedup est quasi-linéaire (x7.73 sur 8 cœurs). Cela indique que notre implémentation a un overhead faible et que la bande passante mémoire DDR5 de la machine suffit à alimenter les 8 cœurs pour cette taille de problème.

4.3 Comparaison sur Charge Irrégulière (Benchmark List)

Ce benchmark simule une charge hétérogène où le temps de calcul de chaque tâche dépend de la valeur du nœud. Nous avons utilisé une liste de 10 nœuds avec des poids croissants de 1 à 10. Le temps séquentiel de référence est de **55 secondes**.

Pour évaluer l'impact de l'ordonnancement, nous avons lancé l'exécution avec **4 workers** selon les deux politiques disponibles. Voici les résultats obtenus :

- **Politique Static (Round-Robin) :**

  ```
  EXPRESSO_WORKER_COUNT=4 EXPRESSO_SCHEDULE=static ./list 10
  ```

  → Temps mesuré : **18.00 s**

- **Politique Balanced (Greedy) :**

  ```
  EXPRESSO_WORKER_COUNT=4 EXPRESSO_SCHEDULE=balanced ./list 10
  ```

  → Temps mesuré : **18.00 s**

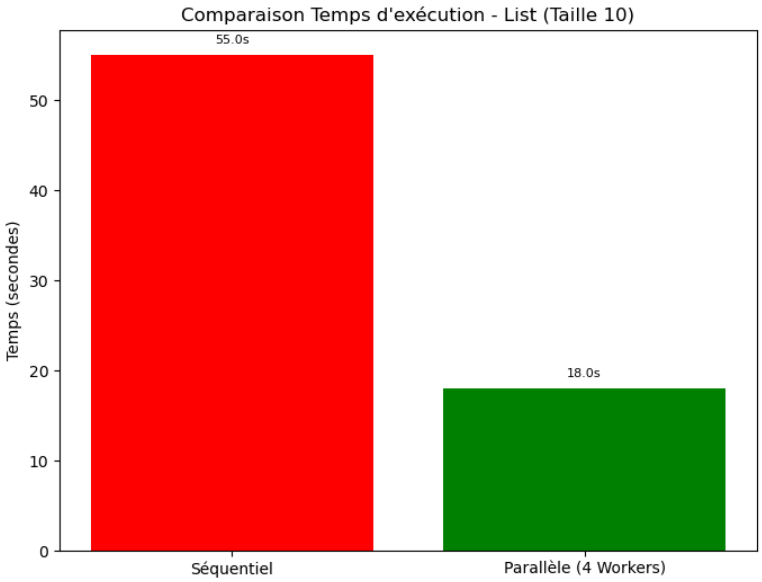


Figure 2: Temps d'exécution sur List (10 nœuds) avec 4 workers.

4.3.0.1 Analyse des résultats : Nous observons un speedup d'environ $\times 3$ par rapport au séquentiel ($55/18 \approx 3.05$). Il est intéressant de noter que les deux politiques obtiennent ici exactement le même temps limite de 18 secondes. Cela s'explique par la structure des données (poids 1..10).

- En mode **Static**, la distribution cyclique sur 4 workers a assigné au worker le plus chargé les tâches de poids 2, 6 et 10, totalisant exactement 18 secondes ($2 + 6 + 10$).
- En mode **Balanced**, l'algorithme d'équilibrage dynamique a convergé vers un temps similaire, limité par la granularité des tâches.

4.3.0.2 Discussion critique : Bien que les performances soient ici identiques, ce résultat est "chanceux" pour le mode *Static*. Sa performance dépend entièrement de l'ordre des tâches. Si la liste avait été triée différemment (par exemple, toutes les tâches lourdes aux indices multiples de 4), le mode *Static* aurait créé un fort déséquilibre. À l'inverse, le mode *Balanced* garantit une répartition de charge optimale (*best-effort*) quel que soit l'ordre d'arrivée des tâches, ce qui en fait la stratégie la plus robuste pour des charges imprévisibles.

5 Conclusion

Ce projet a permis de mettre en œuvre les mécanismes fondamentaux d'un runtime parallèle. Les résultats expérimentaux sur l'architecture Ryzen montrent l'efficacité de l'approche Fork-Join pour des calculs réguliers, avec un passage à l'échelle linéaire jusqu'à 8 cœurs. Pour les applications irrégulières, bien que nos tests sur petits volumes ne départagent pas strictement les politiques, l'architecture hybride mise en place permet théoriquement de mieux gérer le déséquilibre de charge.